

EXPERIMENT:1 Write the python program to Solve 8-Puzzle problem

Aim

To write and execute a Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm.

Objective

To move the tiles of an 8-puzzle from the initial state to the goal state by moving the blank space (0).

Algorithm

1. Start with the initial puzzle configuration.
2. Represent the blank space by 0.
3. Define the goal state as:
4. 1 2 3
5. 4 5 6
7 8 0
6. Add the initial state to a queue.
7. Remove a state from the front of the queue.
8. Find the position of the blank tile.
9. Generate all possible moves by moving the blank tile up, down, left, or right.
10. Add unvisited states to the queue.
11. Continue until the goal state is reached.
12. Display all steps from the initial state to the goal state.

Python Program

```
from collections import deque

# Goal state
GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

# Possible movements of blank space
MOVES = {
    0: [1, 3],
```

```

1: [0, 2, 4],
2: [1, 5],
3: [0, 4, 6],
4: [1, 3, 5, 7],
5: [2, 4, 8],
6: [3, 7],
7: [4, 6, 8],
8: [5, 7]
}

# Display puzzle
def display(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

# BFS algorithm
def solve_puzzle(start):
    queue = deque()
    queue.append((start, [start]))
    visited = set()
    visited.add(start)

    while queue:
        current, path = queue.popleft()

        # Check goal
        if current == GOAL:
            return path

        # Find blank position
        blank = current.index(0)

        # Generate next states
        for position in MOVES[blank]:

```

```

        new_state = list(current)

        # Swap blank and tile
        new_state[blank], new_state[position] = \
            new_state[position], new_state[blank]
        new_state = tuple(new_state)

        # Visit only new states
        if new_state not in visited:
            visited.add(new_state)
            queue.append(
                (new_state, path + [new_state])
            )

    return None

# Initial state
start = (1, 2, 3,
        4, 0, 6,
        7, 5, 8)

# Solve puzzle
solution = solve_puzzle(start)

# Display result
if solution:
    print("Solution Found!")
    print("Number of moves:", len(solution) - 1)
    print("\nSteps:")
    for i, state in enumerate(solution):
        print("Step", i)
        display(state)
else:
    print("No solution exists.")

```

Sample Input

1 2 3

4 0 6

7 5 8

```
8-puzzle problem.py - C:\Users\yadal\OneDrive\Documents\AI\8-puzzle problem.py (3.14.7)
File Edit Format Run Options Window Help

from collections import deque

# Goal state
GOAL = (1, 2, 3,
        |
        4, 5, 6,
        7, 8, 0)

# Possible moves of the blank tile
MOVES = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}

def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()

def bfs(start):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        # Check goal
        if current == GOAL:
            return path

        zero = current.index(0)

        # Generate next states
        for move in MOVES[zero]:
```

Output

```
Enter "help" below or click "Help" above for more information.
>> ===== RESTART: C:\Users\yadal\OneDrive\Documents\AI\8-puzzle problem.py =
Solution found!
Number of moves: 2

Steps:
Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
>> |
```

Result

Thus, the 8-Puzzle problem was successfully solved using the Breadth-First Search algorithm, and the initial state was transformed into the required goal state.